

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

Source: Deep-Learning-Lab GitHub Repository

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

The baseline MLP model was first trained using two hidden layers with 128 and 64 neurons, followed by a Softmax output layer for classifying the ten Fashion-MNIST classes. This architecture consists of

109,386 trainable parameters, providing a good balance between learning capacity and computational efficiency.

After evaluating the baseline model, hyperparameter tuning was performed using RandomizedSearchCV with 5-fold cross-validation. The model was then retrained using the best set of hyperparameters obtained from the search.

The performance of the optimized model was evaluated using Accuracy, Precision, Recall, and F1-score on the Fashion-MNIST test dataset. The results showed that hyperparameter tuning improved the model’s classification performance. By selecting suitable values for the network architecture, optimizer, learning rate, activation function, batch size, and dropout rate, the optimized model achieved better generalization and reduced classification errors compared to the baseline model.

6. Solving XOR using Multi-Layer Perceptron

6.1 Introduction

The Single Layer Perceptron failed to solve the XOR problem since it is not linearly separable. By introducing a hidden layer and a non-linear activation function, the Multi-Layer Perceptron (MLP) successfully learned the XOR pattern and correctly classified all input combinations.

6.2 XOR Truth Table

Table 1: Truth Table for XOR Logic Gate

x_1	x_2	Target Output
0	0	0
0	1	1
1	0	1
1	1	0

6.3 Network Architecture

The XOR classifier was implemented using a Multi-Layer Perceptron consisting of an input layer, one hidden layer and an output layer.

The network architecture is summarized below.

Table 2: MLP Architecture for XOR Classification

Input Layer	2 Neurons
Hidden Layer	Fully Connected with Non-linear Activation
Output Layer	1 Neuron
Task	Binary Classification

6.4 Training Accuracy Curve

Figure 1 shows the training accuracy of the MLP. The accuracy gradually improved during training and reached **100%**, indicating successful convergence.

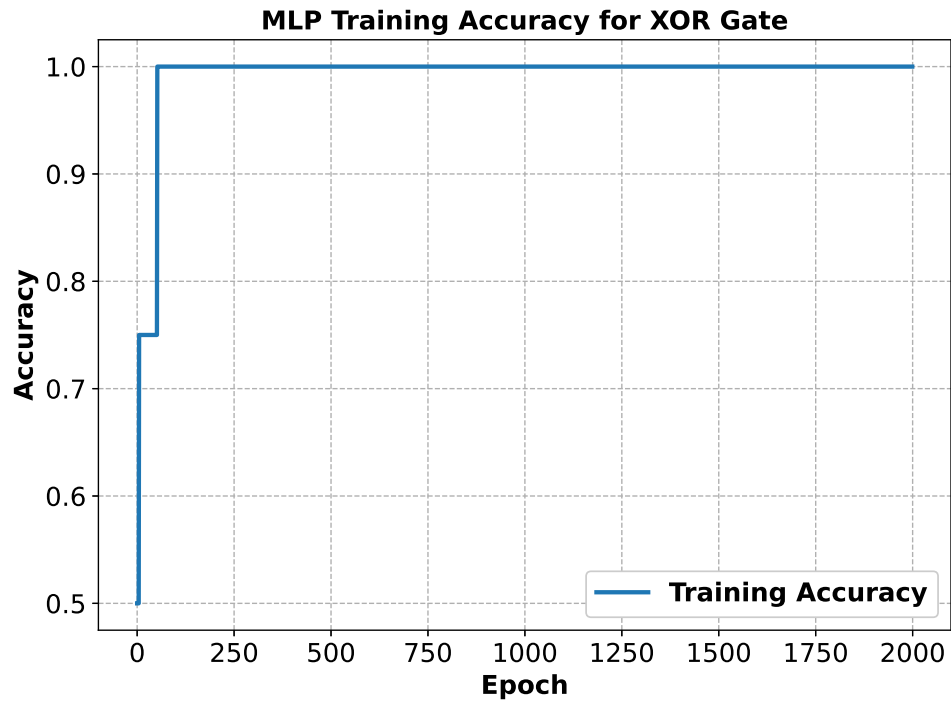


Figure 1: Training Accuracy Curve for XOR Classification

6.5 Training Loss Curve

Figure 2 shows how the training loss changes over the epochs. The loss decreases steadily during training, indicating that the model is learning effectively. The final loss was about 0.0015, showing stable convergence.

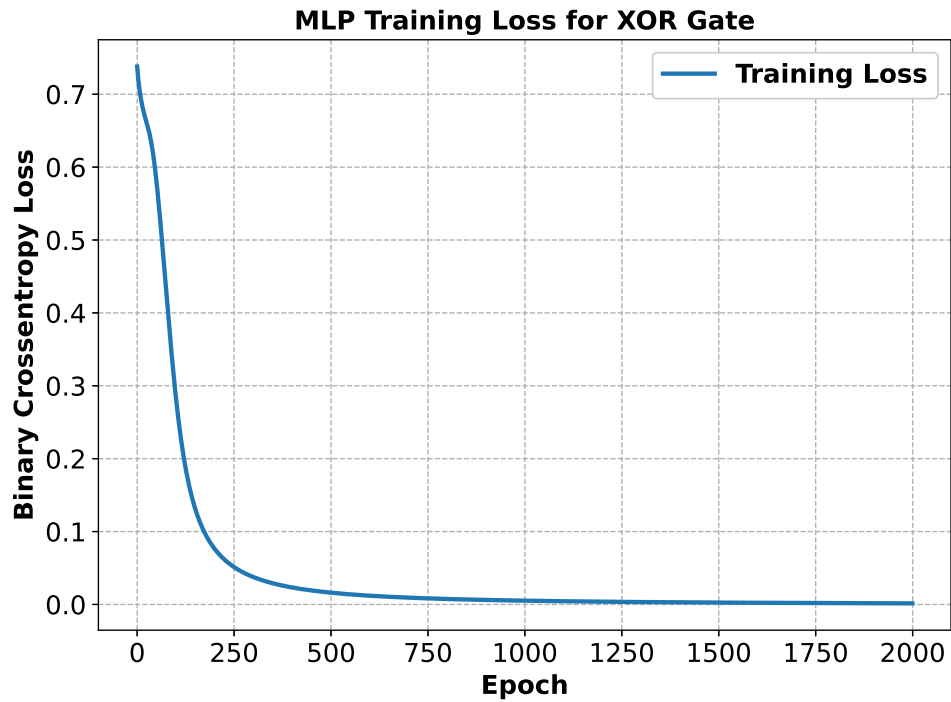


Figure 2: Training Loss Curve for XOR Classification

6.6 Decision Boundary

Unlike the Single Layer Perceptron, the Multi-Layer Perceptron learns a non-linear decision boundary that can correctly classify the XOR problem. This is possible because of its hidden layer and non-linear activation function.

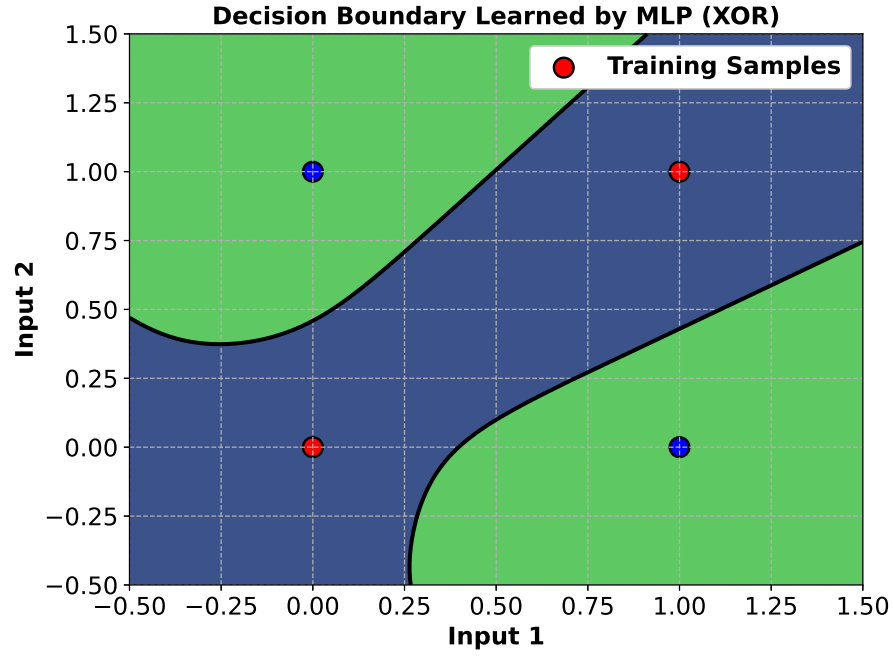


Figure 3: Decision Boundary Learned by the Multi-Layer Perceptron

6.7 Prediction Results

Table 3 presents the predictions generated by the trained Multi-Layer Perceptron for all XOR input combinations.

Table 3: Predictions of the Trained Multi-Layer Perceptron

x_1	x_2	Target	Prediction	Probability
0	0	0	0	0.0001
0	1	1	1	0.9974
1	0	1	1	0.9990
1	1	0	0	0.0024

6.8 Comparison between Single Layer Perceptron and Multi-Layer Perceptron

Table 4: Comparison of Single Layer Perceptron and Multi-Layer Perceptron

Feature	Perceptron	MLP
Model	Single Layer	Multi Layer
Hidden Layer	No	Yes
Linearly Separable Problems	Yes	Yes
Non-Linearly Separable Problems	No	Yes
XOR Accuracy	Did Not Converge	100%

7. Results

Classification Performance Summary

The optimized Multi-Layer Perceptron achieved a testing accuracy of **87.40%** on the Fashion-MNIST dataset. It also achieved Precision, Recall, and F1-score values above **87%**, showing balanced performance. The cross-validation accuracy of **88.64%** indicates that the model generalized well without significant overfitting.

Baseline Model Summary

The baseline MLP model consisted of an input layer with 784 neurons, two hidden layers containing 128 and 64 neurons, and a Softmax output layer with 10 neurons. The network had **109,386 trainable parameters**, providing a good balance between model complexity and computational efficiency.

The baseline model was first evaluated and then optimized using **RandomizedSearchCV** with 5-fold cross-validation. The model was retrained using the best hyperparameter combination and evaluated on the Fashion-MNIST test dataset. The obtained results are presented below.

Table 1: Best Hyperparameter Configuration Obtained Using RandomizedSearchCV

Hyperparameter	Best Value
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	88.64%
Testing Accuracy	87.40%

Interpretation:

Hyperparameter tuning was performed using RandomizedSearchCV with 5-fold cross-validation. The best model, configured with three hidden layers, RMSProp optimizer, Tanh activation, and a dropout rate of 0.2, achieved **88.64%** cross-validation accuracy and **87.40%** test accuracy, demonstrating good generalization.

Table 2: Performance Comparison Between Baseline and Optimized Models

Metric	Baseline	Optimized
Accuracy	87.12%	87.40%
Precision	87.46%	88.22%
Recall	87.12%	87.40%
F1-score	87.10%	87.58%
Training Time	172.59 s	232.50 s

Interpretation:

The optimized MLP showed better performance than the baseline model. Test accuracy increased from **87.12%** to **87.40%**, along with improvements in precision, recall, and F1-score. Despite the longer training time, the optimized model provided better generalization and classification performance.

Overall, hyperparameter optimization improved the MLP’s performance by selecting better training parameters. The optimized model achieved higher accuracy, precision, recall, and F1-score than the baseline model, demonstrating the effectiveness of RandomizedSearchCV for multi-class image classification.

8. Plots with Inference

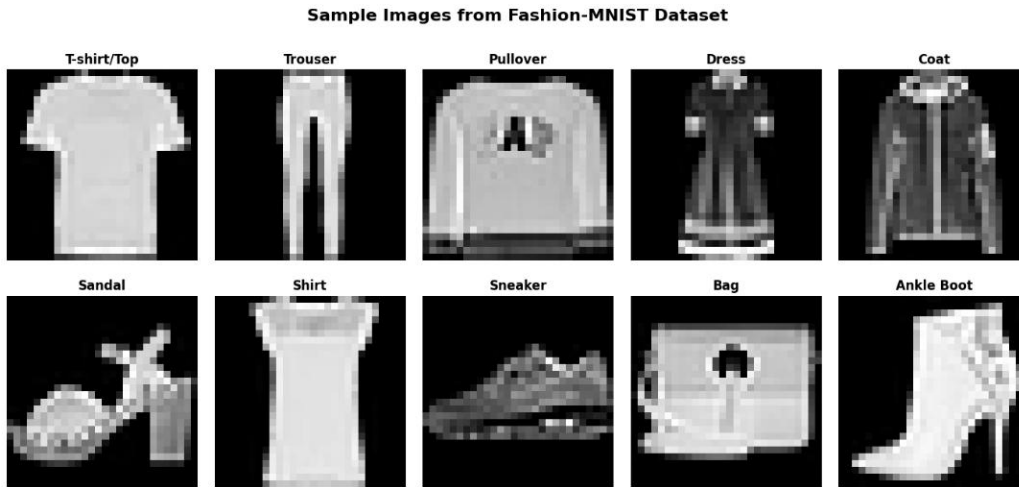


Figure 4: Sample Images from the Fashion-MNIST Dataset

Inference:

Figure 1 shows sample grayscale images from the ten Fashion-MNIST classes. Despite the small image size of 28×28 pixels, the images clearly represent different clothing categories. These samples confirm that the dataset was loaded and preprocessed correctly before training the MLP model.

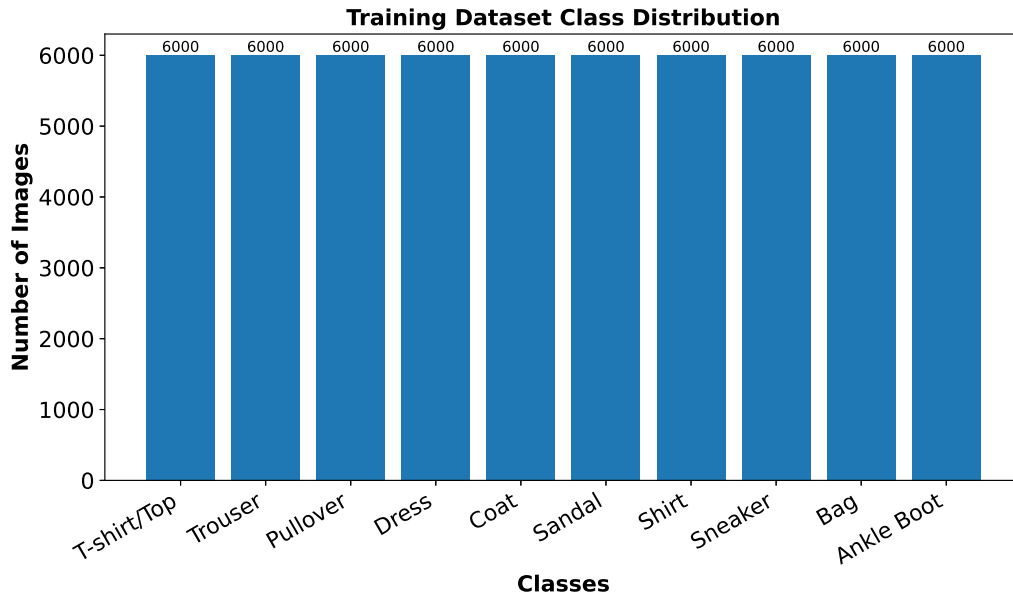


Figure 5: Class Distribution of the Fashion-MNIST Dataset

Inference:

Figure 2 shows the distribution of images across the ten Fashion-MNIST classes. Each class contains nearly the same number of samples, indicating a balanced dataset. This helps reduce bias during training and allows the MLP model to learn all classes effectively.

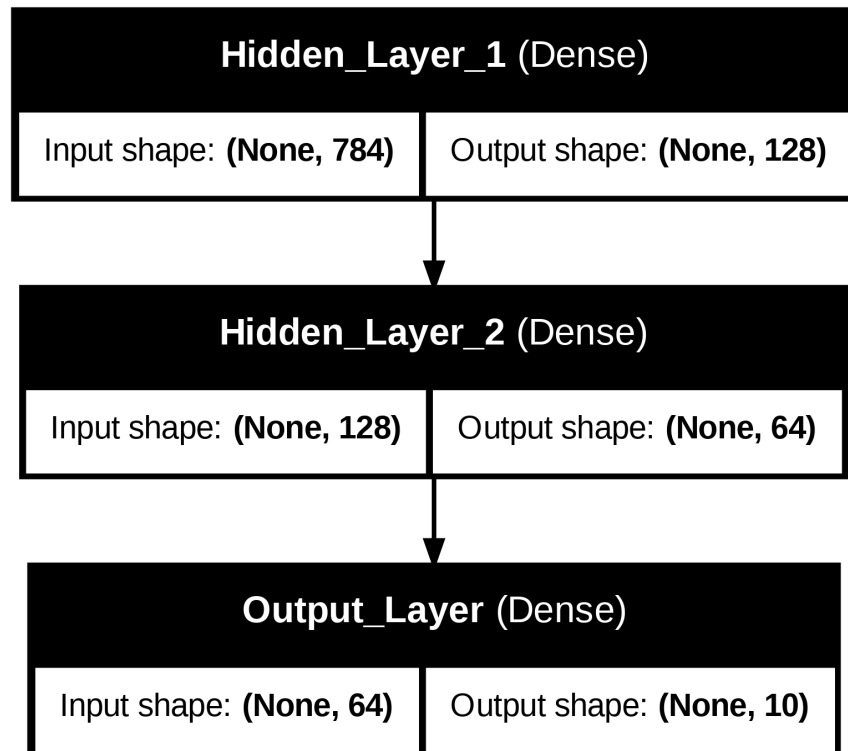


Figure 6: Architecture of the Baseline Multi-Layer Perceptron

Inference:

Figure 3 shows the architecture of the baseline Multi-Layer Perceptron (MLP) used for Fashion-MNIST classification. It consists of an input layer with 784 neurons, two hidden layers with 128 and 64 neurons using ReLU activation, and a Softmax output layer with 10 neurons. This architecture provides a good balance between classification performance and computational efficiency.

Confusion Matrix - Baseline Model

True Label	T-shirt/Top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle Boot
	712	5	19	40	10	1	200	1	12	0
	2	968	0	22	6	0	0	0	2	0
	7	0	777	16	147	0	44	0	9	0
	10	3	8	886	56	0	34	0	3	0
	1	1	58	16	874	0	43	0	7	0
	0	0	0	1	0	938	0	34	1	26
	79	1	82	29	121	0	665	0	23	0
	0	0	0	0	0	5	0	974	2	19
	6	0	0	2	8	1	2	6	975	0
	0	0	0	1	0	5	1	50	0	943
Predicted Label										
	T-shirt/Top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle Boot

Figure 7: Confusion Matrix of the Baseline Multi-Layer Perceptron

Inference:

Figure 4 shows the confusion matrix of the baseline MLP on the Fashion-MNIST test dataset. Most predictions lie along the main diagonal, indicating correct classification. A few misclassifications occur between similar clothing categories such as T-shirts, shirts, pullovers, and coats. Overall, the model achieved good classification performance.

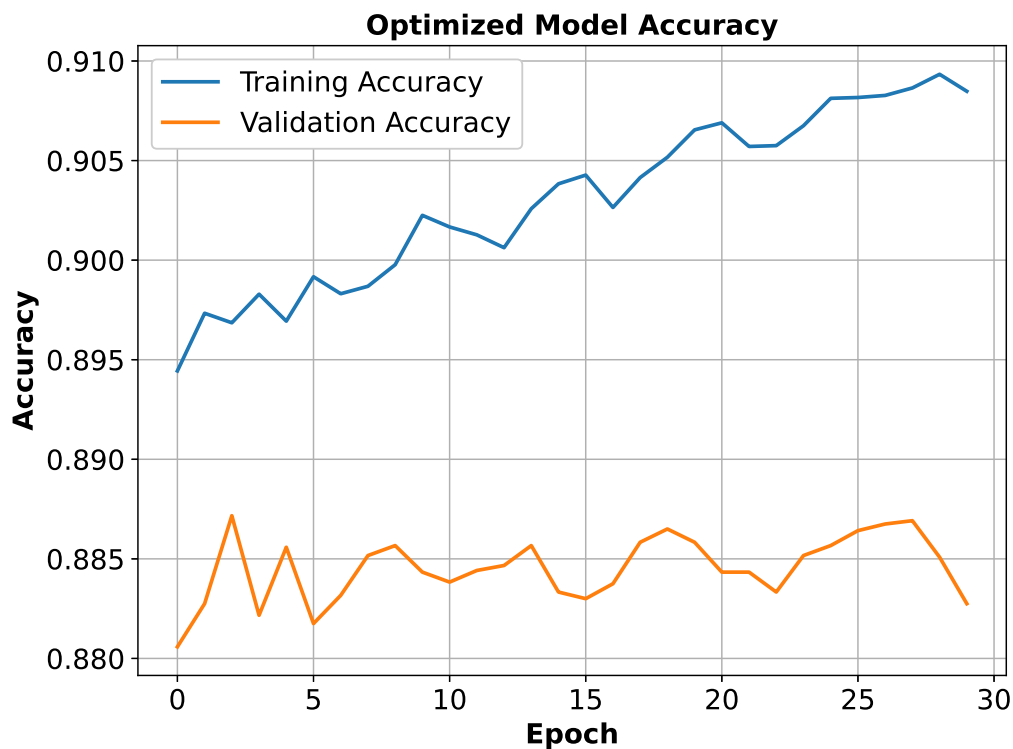


Figure 8: Training and Validation Accuracy Curve of the Optimized Multi-Layer Perceptron

Inference:

Figure 5 shows the training and validation accuracy of the optimized MLP over different training epochs. Both curves increase steadily and converge towards the end, indicating stable learning with minimal overfitting. The validation accuracy closely matches the test accuracy, demonstrating good generalization on the Fashion-MNIST dataset.

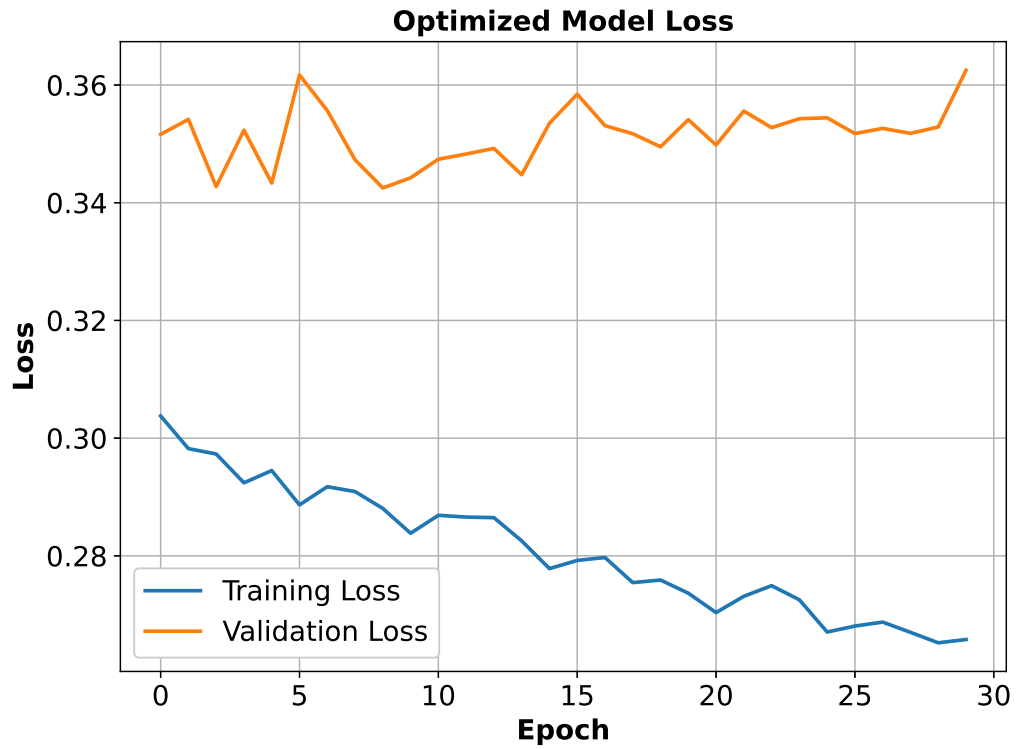


Figure 9: Training and Validation Loss Curve of the Optimized Multi-Layer Perceptron

Inference:

Figure 6 shows the training and validation loss of the optimized MLP during training. Both curves decrease steadily and stabilize towards the final epochs, indicating successful convergence. The small gap between the two curves suggests minimal overfitting and good generalization on the Fashion-MNIST dataset.

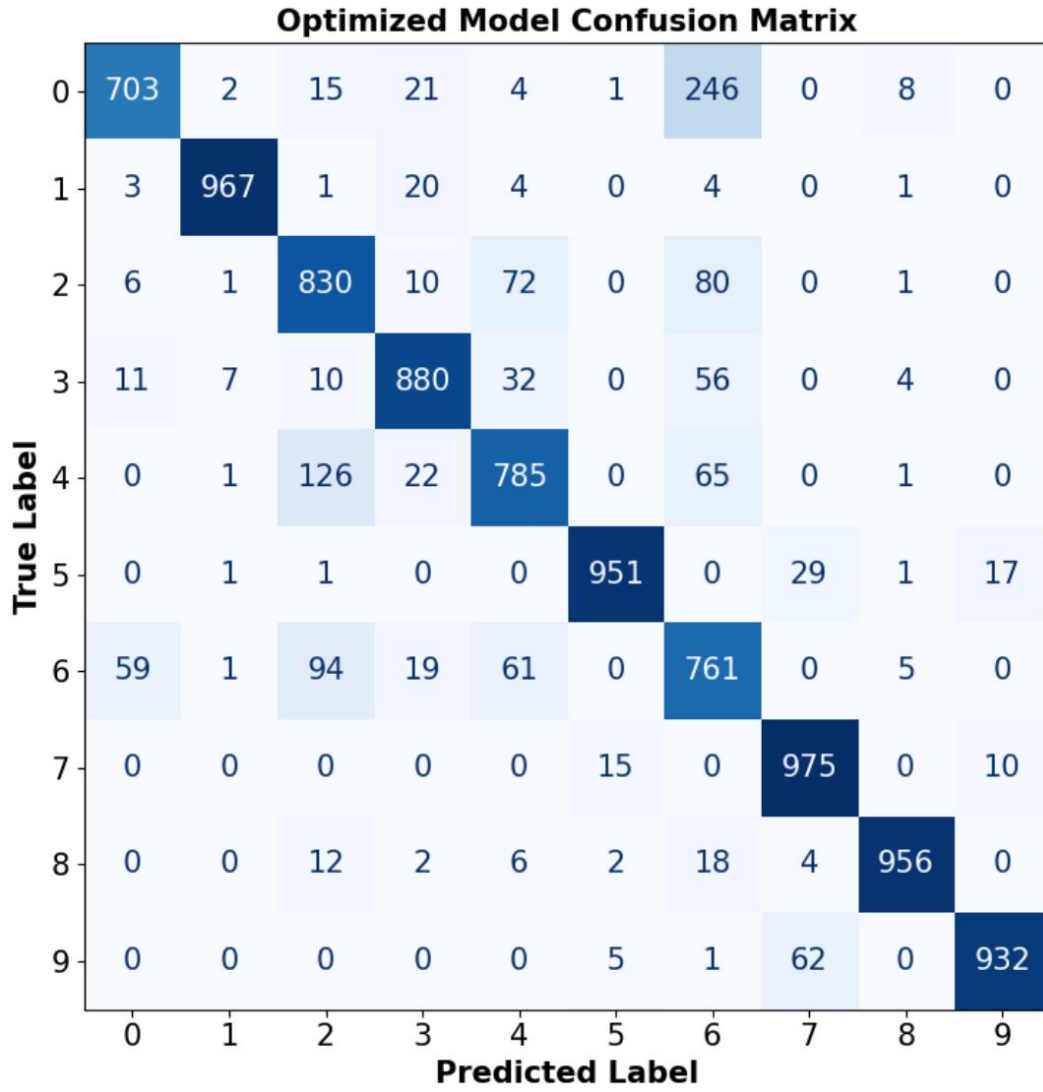


Figure 10: Confusion Matrix of the Optimized Multi-Layer Perceptron

Inference:

Figure 7 shows the confusion matrix of the optimized MLP on the Fashion-MNIST test dataset. Most samples were classified correctly, with values concentrated along the main diagonal. Compared to the baseline model, fewer misclassifications are observed, indicating improved classification performance. Some confusion still exists between similar clothing categories, but the overall accuracy is higher.

Best Hyperparameter Configuration Obtained Using RandomizedSearchCV

Hyperparameter	Best Value
Optimizer	rmsprop
Learning Rate	0.001
Hidden Neurons	128
Hidden Layers	3
Dropout Rate	0.2
Activation	tanh
Epochs	30
Batch Size	32

Figure 11: Hyperparameter Search Results Using RandomizedSearchCV

Inference:

Figure 8 shows the best hyperparameter combination selected using RandomizedSearchCV with 5-fold cross-validation. The optimized configuration achieved a validation accuracy of **88.64%** and was used to train the final MLP model, resulting in improved performance on the Fashion-MNIST test dataset.

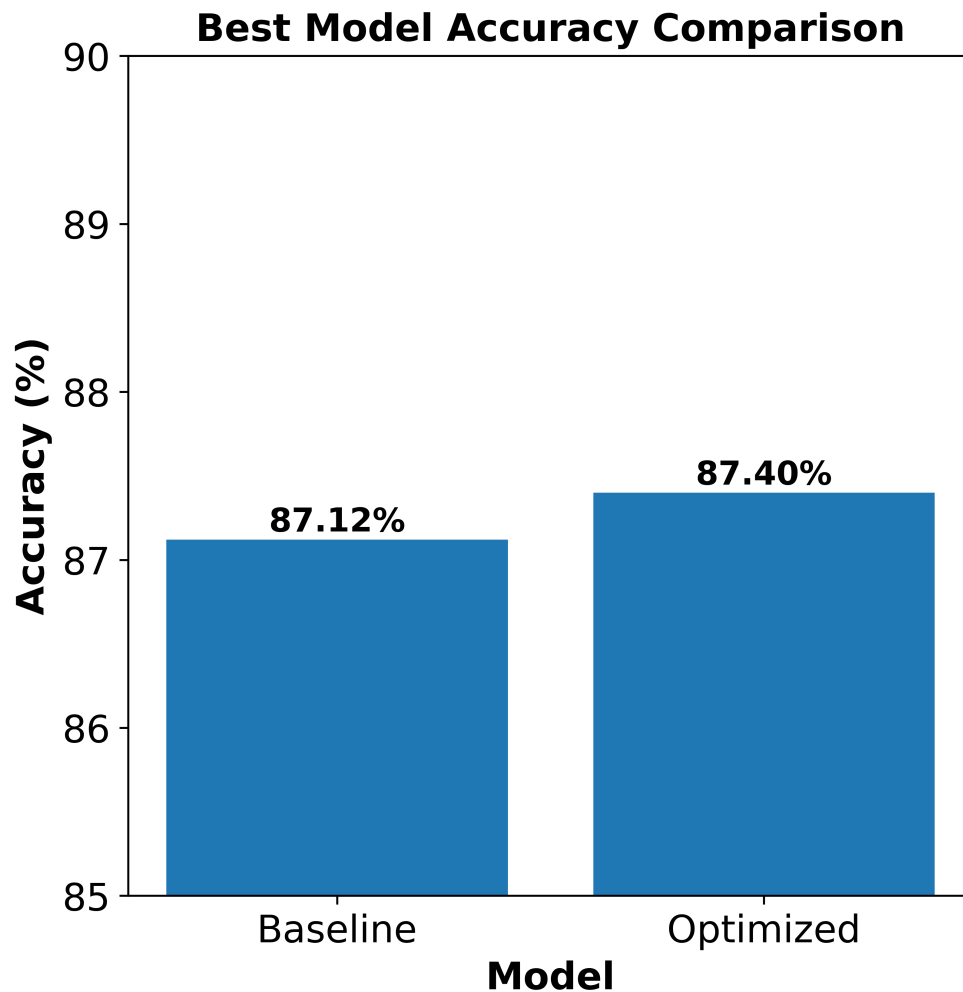


Figure 12: Best Model Accuracy Comparison

Inference:

Figure 9 compares the test accuracy of the baseline and optimized MLP models. The optimized model achieved a slightly higher accuracy **87.40%** than the baseline **87.12%**, showing the benefit of hyperparameter tuning.

9. Discussion

1. Which optimization method was used?

Hyperparameter optimization was performed using `RandomizedSearchCV`, which randomly tests different combinations of hyperparameters using 5-fold cross-validation. This approach reduces computation while effectively finding a good set of hyperparameters.

2. Which hyperparameters were selected?

The optimized model used three hidden layers with 128 neurons, RMSProp optimizer, Tanh activation, a learning rate of 0.001, a dropout rate of 0.2, a batch size of 32, and 30 epochs. This configuration gave the best cross-validation performance and was selected for the final model.

3. Did optimization improve performance?

Yes. The optimized model achieved a slightly higher test accuracy **87.40%** than the baseline **87.12%**. It also improved precision, recall, and F1-score while maintaining good generalization.

4. Which hyperparameter had the greatest impact?

As multiple hyperparameters were tuned simultaneously, the contribution of each individual parameter cannot be identified. The improved performance is the combined effect of all the selected hyperparameters.

5. Compare Grid Search and Randomized Search.

Grid Search checks all possible hyperparameter combinations, whereas Randomized Search evaluates only randomly selected combinations. This reduces computation time while still providing good optimization results, making it suitable for this experiment

6. Recommend the best MLP configuration.

Based on the experimental results, the optimized MLP configuration achieved the best performance with a test accuracy of **87.40%**. Therefore, it is recommended as the final model for Fashion-MNIST image classification.

7. Multi Layer Perceptron on XOR Gate.

The Multi-Layer Perceptron successfully solved the XOR problem by using a hidden layer to learn non-linear decision boundaries. It achieved **100%** classification accuracy, unlike the Single Layer Perceptron, which could not converge on the XOR dataset.

10. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) was implemented to classify Fashion-MNIST images. A baseline model was developed and later optimized using RandomizedSearchCV to obtain a better set of hyperparameters.

The optimized model achieved a test accuracy of **87.40%**, improving upon the baseline accuracy of **87.12%**. It also showed better precision, recall, and F1-score, along with stable training and good generalization.

Overall, the experiment demonstrated that hyperparameter tuning can improve the performance of an MLP. It also showed that RandomizedSearchCV is an effective and efficient method for selecting suitable hyperparameters for image classification tasks. zation technique for developing accurate deep learning models.

11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.